

LAB# 4

Combine Information from Multiple MySQL Tables with JOIN

You can use a JOIN SELECT query to combine information from more than one MySQL table. With JOIN, the tables are combined side by side, and the information is retrieved from both tables.

As an example of inner and outer joins, consider a Clothes catalog with two tables. One table is Product, with the two columns Name and Type holding the following data:

Name	Type
T-shirt	Shirt
Dress shirt	Shirt
Jeans	Pants

The second table is Color, with two columns Name and Color holding the following data:

Name	Color
T-shirt	white
T-shirt	red
Loafer	black

You need to ask a question that requires information from both tables. If you do an inner join with the following query:

```
SELECT * FROM Product,Color WHERE Product.Name = Color.Name
```

You get the following results table with four columns: Name (from Product), Type, Name (from Color), and Color.

Name	Type	Name	Color
T-shirt	Shirt	T-shirt	white
T-shirt	Shirt	T-shirt	red

Notice that only T-shirt appears in the results table

Do a left outer join with the following query:

```
SELECT * FROM Product LEFT JOIN Color  
ON Product. Name=Color. Name
```

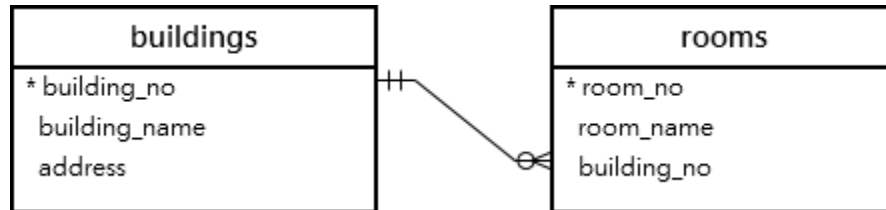
You get the following results table, with the same four columns — Name (from Product), Type, Name (from Color), and Color — but with different rows:

Name	Type	Name	Color
T-shirt	Shirt	T-shirt	white
T-shirt	Shirt	T-shirt	red
Dress shirt	Shirt		
Jeans	Pants		

This table has four rows. It has the same first two rows as the inner join, but it has two additional rows — rows that are in the Product table on the left but not in the Color table. Notice that the columns from the table Color are blank for the last two rows.

MySQL ON DELETE CASCADE example

The relationship between the buildings and rooms tables is one-to-many (1:N) as illustrated in the following database diagram:



The following are steps that demonstrate how the ON DELETE CASCADE referential action works.

Step 1. Create the buildings table:

Step 2. Create the rooms table:

```
CREATE TABLE rooms (  
    room_no INT PRIMARY KEY AUTO_INCREMENT,  
    room_name VARCHAR(255) NOT NULL,  
    building_no INT NOT NULL,  
    FOREIGN KEY (building_no)  
        REFERENCES buildings (building_no)  
        ON DELETE CASCADE  
);
```

Notice that the ON DELETE CASCADE clause at the end of the foreign key constraint definition.

Step 3. Insert rows into the buildings table:

```
INSERT INTO buildings(building_name,address)  
VALUES('ACME Headquarters','3950 North 1st Street CA 95134'),  
      ('ACME Sales','5000 North 1st Street CA 95134');
```

Step 4. Query data from the buildings table:

```
SELECT * FROM buildings;
```

	building_no	building_name	address
▶	1	ACME Headquarters	3950 North 1st Street CA 95134
	2	ACME Sales	5000 North 1st Street CA 95134

Step 5. Insert rows into the rooms table:

```
INSERT INTO rooms(room_name,building_no)  
VALUES('Amazon',1),  
      ('War Room',1),  
      ('Office of CEO',1),  
      ('Marketing',2),  
      ('Showroom',2);
```

Step 6. Query data from the rooms table:

SELECT * FROM rooms;

	room_no	room_name	building_no
▶	1	Amazon	1
	2	War Room	1
	3	Office of CEO	1
	4	Marketing	2
	5	Showroom	2

We have three rooms that belong to building no 1 and two rooms that belong to the building no 2.

Step 7. Delete the building with building no. 2:

DELETE FROM buildings

WHERE building_no = 2;

Step 8. Query data from rooms table:

SELECT * FROM rooms;

	room_no	room_name	building_no
▶	1	Amazon	1
	2	War Room	1
	3	Office of CEO	1

As you can see, all the rows that reference to building_no 2 were automatically deleted.

ON UPDATE cascade

This is the same as ON DELETE cascade. The difference is that instead of delete referenced table data it will update the data.

So we follow the same steps as ON DELETE cascade.

1. Create person table.
2. Create orders table with ON UPDATE cascade.
3. Insert data in both the tables.
4. Now update data from person table.
5. Now select data from both the tables and see the difference.

Follow the above steps:

person Table

- 1 create table person(personId bigint(12) primary key,
- 2 name varchar(20),
age int(3));

3

2- orders Table

```
1 create table orders(orderId bigint(12) primary key, orderName varchar(20),
2   personId bigint(20),
3   foreign key(personId) references person(personId)
4   ON UPDATE CASCADE );
```

3- Insert Data in both tables.

```
1 insert into person values(1,"vasu",26);
2 insert into person values(2,"Ayush",26);
3 insert into person values(3,"sushmits",25);
4
5 insert into orders values(1,"Shoes", 1);
6 insert into orders values(2,"Jeans", 1);
7 insert into orders values(3,"Tshirt", 3);
8 insert into orders values(4,"Jacket",2 );
```

4- Update personId from person table.

```
1 update person set personId = 4 where personId = 1;
```

5- Now see the difference by making select query on both the tables.